

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 7. String and its Methods

Lecture: 70. Indexing Slicing

Section 1: Lecture Summary

This lecture covers two fundamental Python string operations: indexing and slicing. It begins by revising string indexing, explaining how to access characters in a string using positive and negative indices with square brackets. The lecture emphasizes that strings are immutable objects, meaning their contents cannot be altered once created. Next, it introduces the concept of string slicing, which uses the same square bracket operator with colon-separated start, stop, and step parameters to extract substrings. The relationship between slicing and Python's range function (start, stop, step semantics) is discussed in detail. The lecturer demonstrates how to use positive and negative indices and steps, including reversing strings using a negative step. The lecture concludes with practical examples executed in Jupyter Notebook, encouraging students to practice these operations extensively to build intuition.

Section 2: Key Concepts and Explanations

- String Indexing
 - Strings are sequences of characters accessible via indices.
 - Indices start at 0 for the first character (left to right).
 - Negative indices start at -1 for the last character (right to left).
 - Access syntax: `string[index]` where `index` can be positive or negative.
 - Example: `s` yields `'H'` in `'Hello World'`.
 - Strings are immutable: you cannot change characters directly (e.g., `s = 'M'` is invalid).
- Immutable Strings

- Once a string is created, its characters and length cannot be changed.
- Any "modification" creates a new string object instead.
- This immutability is common among many Python data types.

- String Slicing

- Uses syntax ``string[start:stop:step]`` inside square brackets.
- ``start`` is the beginning index (inclusive).
- ``stop`` is the ending index (exclusive).
- ``step`` determines the sampling interval (default is 1).
- Similar to the parameters in ``range(start, stop, step)``.

- Behavior and Rules of Slicing

- If ``start`` is omitted, slicing starts at index 0.
- If ``stop`` is omitted, slicing goes up to the end of the string.
- If ``step`` is omitted, it defaults to 1 (forward direction).
- The result is a new substring (new string object).
- Negative indices are allowed for ``start`` and ``stop``.
- The ``start`` index must precede the ``stop`` index in the direction of the step.
- Using a negative ``step`` slices the string backwards, reversing the order.
- When stepping backwards, ``start`` should be greater than ``stop`` (indices adjusted accordingly).

- Use of Negative Step for Reversal

- Example: ``string[::-1]`` creates a reversed copy of the string.
- Allows flexible extraction of substrings in reverse order.

- Application Beyond Strings
- Indexing and slicing apply to other sequence types like lists and tuples.
- Mastery of string slicing helps understand slicing across Python sequences.

Section 3: Example Code and Use Cases

Creating a string

Indexing examples

Attempt to modify string (will raise an error)

`s1[0] = 'M'` `TypeError: 'str' object does not support item assignment`

Slicing examples

Simple slicing from index 3 to 7 (excluding 7)

Slicing from start to index 7

Slicing from index 6 to end

Using negative indices in slicing

Slicing with step

Full string slice - returns entire string

Reversing string using negative step

Slicing with negative step and indices

Another negative step example

```
s1 = "Hello World"

print(s1[0])      # Output: 'H'
print(s1[-7])     # Output: 'o' (7th from the end)

print(s1[3:7])    # Output: 'lo W'

print(s1[:7])     # Output: 'Hello W'
```

```
print(s1[6:])      # Output: 'World'

print(s1[-8:-4])   # Output: 'lo W'

print(s1[2:7:2])   # Output: 'lo'

print(s1[:])       # Output: 'Hello World'

s2 = s1[::-1]
print(s2)          # Output: 'dlrow olleH'

s3 = s1[-7:-12:-1]
print(s3)          # Output: 'olleH'

s4 = s1[10:5:-1]
print(s4)          # Output: 'dlrow'
```

Explanation:

- `s1[3:7]` extracts characters starting at index 3 up to (but not including) 7.
- Omitting start or stop defaults to start of string or end of string, respectively.
- Steps control the stride; `2` means every second character.
- `[::-1]` reverses the string by stepping backward one character at a time.
- Negative step slicing requires careful ordering of start and stop indices in reverse direction.

Section 4: Key Takeaways

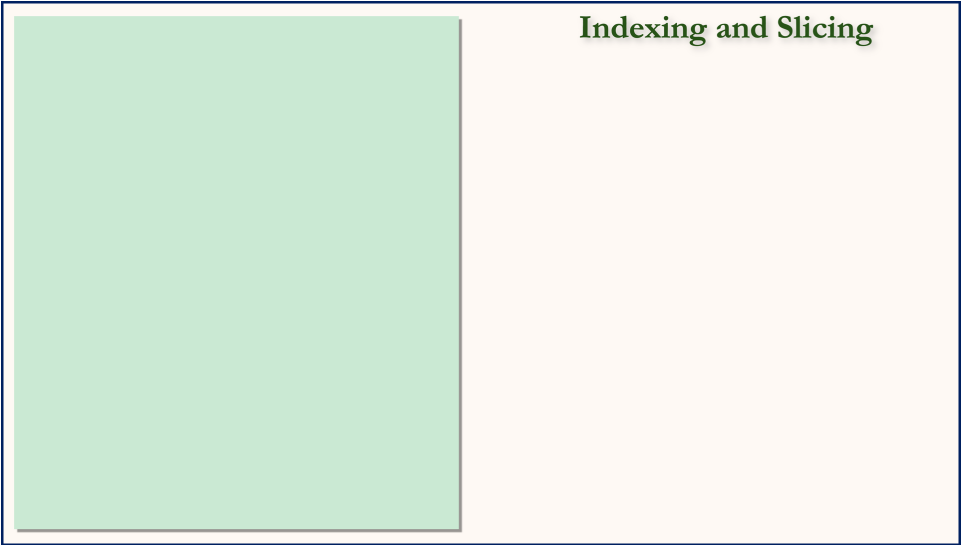
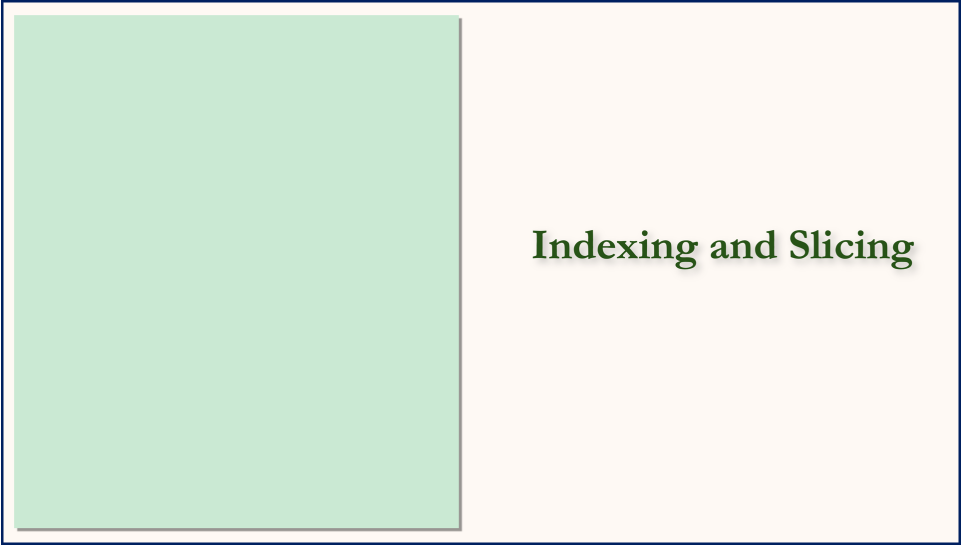
- Indexing allows accessing single characters via positive or negative integer positions.
- Strings in Python are immutable; you cannot change them in place.
- Slicing extracts substrings using the `[start:stop:step]` syntax with similar semantics to the range function.
- Start is inclusive; stop is exclusive; step controls the increment or decrement between indices.

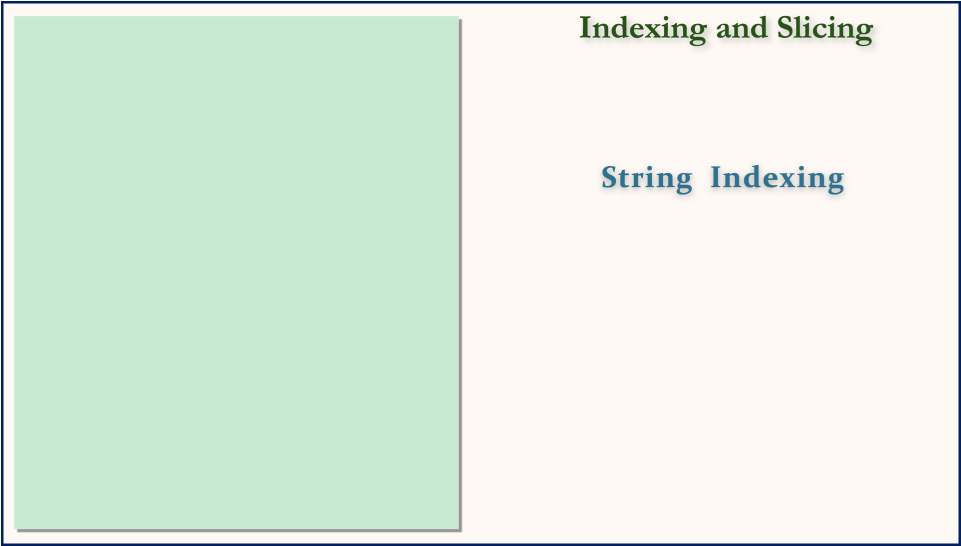
- Default values exist: start=0, stop=length of string, step=1 when omitted.
- Negative indices count backwards from the end of the string.
- Negative steps slice the string in reverse order and can be used for reversing strings.
- The slicing operation returns a new string object without modifying the original string.
- The principles of indexing and slicing extend beyond strings to other Python sequences like lists and tuples.
- Mastery of indexing and slicing enhances ability to manipulate sequences efficiently in Python.
- Practice by experimenting with various start, stop, and step values to visualize their effects.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.

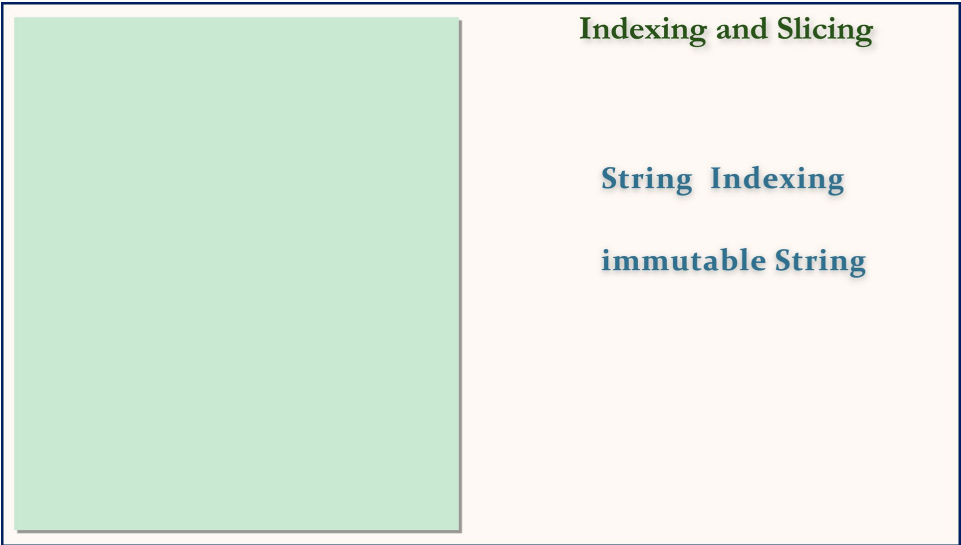






Indexing and Slicing

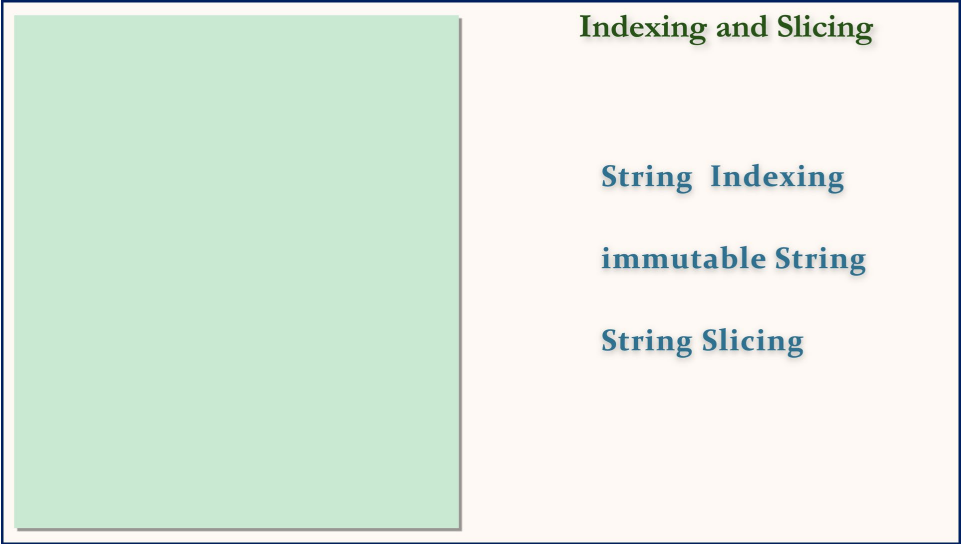
String Indexing



Indexing and Slicing

String Indexing

immutable String

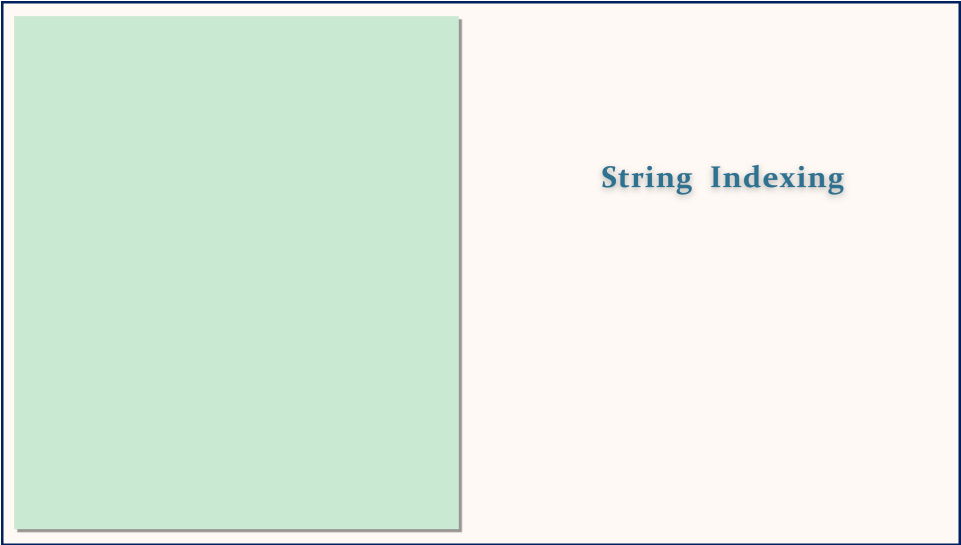


Indexing and Slicing

String Indexing

immutable String

String Slicing



String Indexing

String Indexing

String Indexing

S1 = 'Hello World'

String Indexing

S1 = 'Hello World'

S1	0	1	2	3	4	5	6	7	8	9	10
	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
	H	e	l	l	o		W	o	r	l	d

String Indexing

S1 = 'Hello World'

S1	0	1	2	3	4	5	6	7	8	9	10
	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
	H	e	l	l	o		W	o	r	l	d

String Indexing

S1 = 'Hello World'

	0	1	2	3	4	5	6	7	8	9	10
S1	H	e	l	l	o		W	o	r	l	d
	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

H ← print(S1[0])

String Indexing

S1 = 'Hello World'

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

H ← print(S1[0])

String Indexing

S1 = 'Hello World'

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

H ← print(S1[0]) o ← print(s1[-7])

String Indexing

S1 = 'Hello World'

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[0] = 'H'

String Indexing

S1 = 'Hello World'

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[0] ~~=~~ 'H'

String Indexing

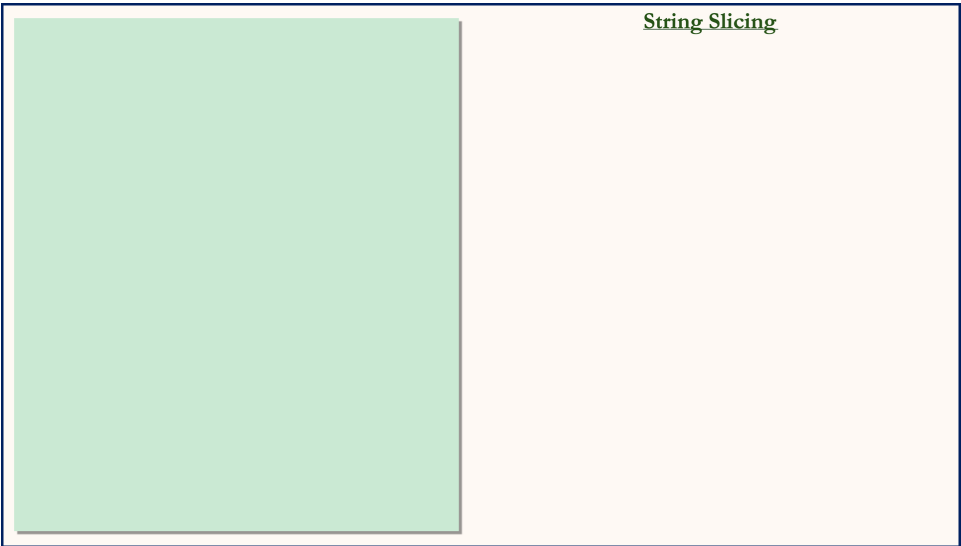
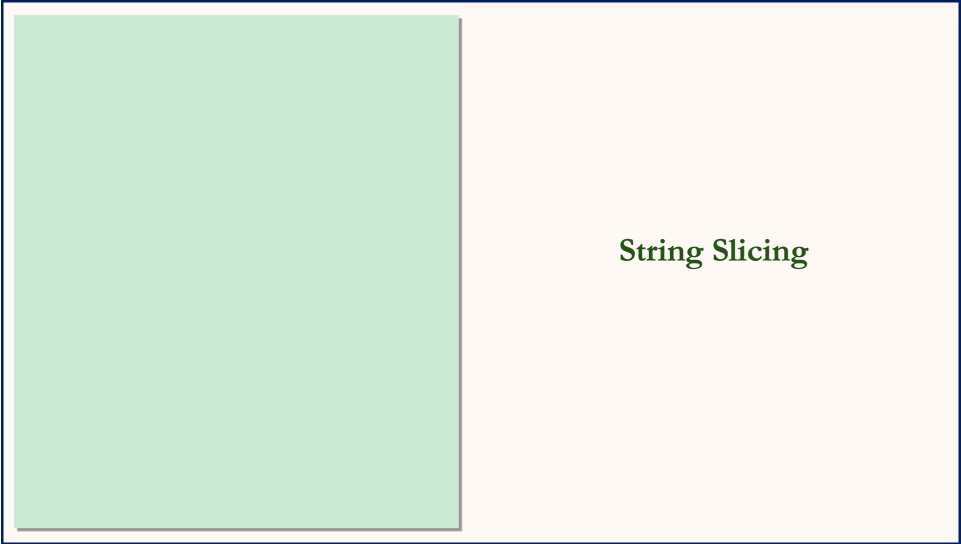
S1 = 'Hello World'

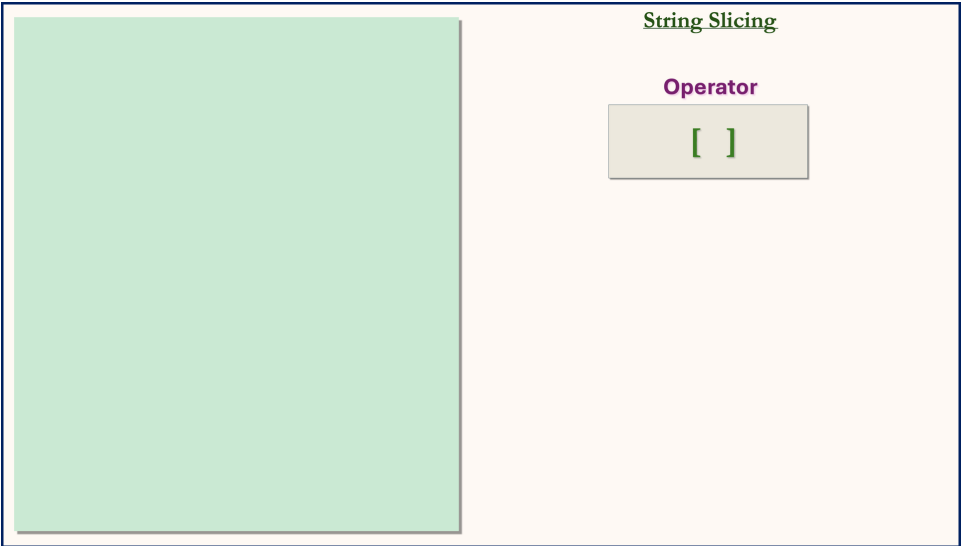
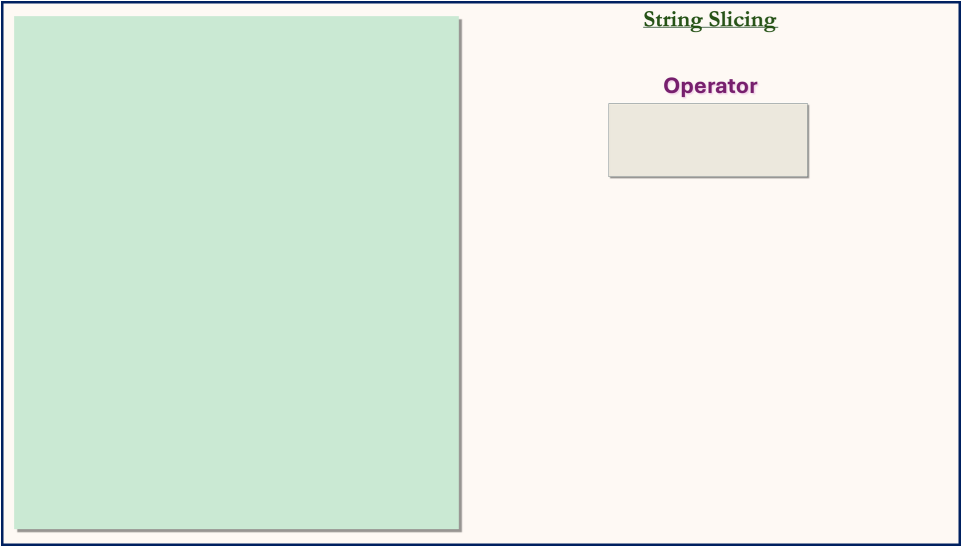
S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[0] ~~=~~ 'M'

Immutable





String Slicing

Operator

S1[]

String Slicing

Operator

S1[index]

String Slicing

Operator

S1[start]

String Slicing

Operator

S1[start : stop]

String Slicing

Operator

S1[start : stop : step]

String Slicing

Operator

S1[start : stop : step]

range(start , stop , step)

String Slicing

Operator

S1[start : stop : step]

range(1 , 10 , step)

String Slicing

Operator

S1[start : stop : step]

range(1, 10, 2)

[1, 3, 5, 7, 9]

String Slicing

Operator

S1[start : stop : step]

string

String Slicing

Operator

S1[start : stop : step]

stringlist

String Slicing

Operator

S1[start : stop : step]

stringlisttuple

String Slicing

Operator

S1 [start : stop : step]

Sequence

string list tuple

String Slicing

Operator

S1[start : stop : step]

Sequence

stringlisttuple

seq

String Slicing

Operator

S1[start : stop : step]

Sequence

stringlisttuple

seq []

String Slicing

Operator

S1[start : stop : step]

Sequence

stringlisttuple

seq[start : stop : step]

String Slicing

S1[start : stop : step]

String Slicing

S1[start : stop : step]

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[start : stop : step]

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[start]: stop : step

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[6]: stop : step

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[start : stop] step

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[1 : stop] step

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[1 : 7] : step

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o	W	o	r	l	d	
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[3 : 7] : step

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o	W	o	r	l	d	
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[: 7] : step

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[:] : step

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[6 :] : step

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[-5 :-2] : step

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[-11: -2] : step

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1

[0 : stop : step]

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1

[0 : 11 : step]

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[0 : 11 : 2]

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[0 : : 2]

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S1[: : 2]

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S2 = S1[: :]

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S2

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S2 = S1[: :]

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S2

0	1	2	3	4	5	6	7	8	9	10
d	l	r	o	W		o	l	l	e	H
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

```
S2 = S1[ : :-1 ]
```

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S2

0	1	2	3	4	5	6	7	8
r	o	W		o	l	l	e	H
-9	-8	-7	-6	-5	-4	-3	-2	-1

```
S2 = S1[ -3 : :-1 ]
```

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S2

0	1	2	3	4	5
r	o	W		o	l
-6	-5	-4	-3	-2	-1

```
S2 = S1[ -3 : -9 : -1 ]
```

String Slicing

S1

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

S2

0	1	2	3	4	5
r	o	W		o	l
-6	-5	-4	-3	-2	-1

```
S2 = S1[ 8 : 2 : -1 ]
```

